ATWE.

IOS ARCHITECTURE & BUILD PLAN

A native iPhone app, unmistakably Atwe.

How we turn the existing Atwe web platform into a production-quality native iOS app (and, from the same codebase, Android) — reusing the entire existing backend, preserving the five-world product, and earning a true Apple-grade experience. Includes the technology decision and its trade-offs, the repository strategy, the full architecture, a phase-by-phase build roadmap, and App Store readiness.

Expo + TypeScript

Expo Router native nav

Dedicated atwe-mobile repo

Reuses the **existing** backend

iOS **+ Android** one codebase

OVERVIEW

Executive summary & guiding principles

The goal is not to redesign Atwe. It is to give Atwe a native iPhone body while keeping its soul — the same five worlds, the same identity, the same backend — and to make it feel like an app Apple would ship. This plan commits to **Expo + TypeScript + Expo Router** in a **dedicated `atwe-mobile` repository** that talks directly to your existing Express + PostgreSQL backend on Railway. Nothing on the server changes.

The seven priorities, and how the architecture serves each

YOUR PRIORITY	HOW THIS PLAN DELIVERS IT
Keep the Atwe identity	The web CSS variables are ported 1:1 into a native token system (already done). "White acts, blue identifies", the silver verified seal, Black/Light themes — all encoded, so the app is recognizably Atwe on first launch.
Feel like a true iPhone app	Expo Router gives native stack navigation, the edge-swipe back gesture, and native transitions for free. Haptics, blur materials, native sheets, safe-area/Dynamic-Island handling, and Dynamic Type are first-class.
Follow Apple's HIG	A dedicated HIG checklist (navigation, gestures, sheets, context menus, safe areas, Face ID, Dynamic Type, VoiceOver, light/dark) is baked into the component layer and reviewed each phase.
Premium experience	Subtle, confident motion (Reanimated), Liquid-Glass translucency used sparingly, every pixel purposeful — no flashy effects.
Preserve all backend functionality	The app is a pure client of the existing REST + SSE API. A typed API-contract layer maps the ~470-endpoint surface. Zero duplicated business logic.
Extremely high code quality	TypeScript strict mode, a clean layered architecture (UI · hooks · services · api · state), ESLint + Prettier, and small, single-purpose modules.
Scale to millions	Expo/EAS is proven at scale (Discord, Shopify, Coinbase all run React Native). The design keeps rendering lean, caches aggressively (React Query), and relies on APNs — not open sockets — for background delivery.

> **Status.** Phase 0 (this foundation) is **already built, committed and pushed**: project config, the design-token system, the typed API + realtime clients, secure Keychain auth with the 2FA challenge, and the five-world tab navigation with a real login and a real Profile screen. You can run it on your iPhone today via Expo Go. The rest of this document is the decisions behind it and the road ahead.

ARCHITECTURE DECISION 01

Why Expo + TypeScript (and not the alternatives)

You asked for React Native via Expo "unless there is a technically superior reason to document." There isn't — Expo is the right call. Here is the reasoning, the alternatives weighed, and the trade-offs, so the decision is on the record.

✔ Chosen — Expo (managed) + TypeScript + Expo Router

One codebase → iOS + Android. Native navigation & gestures out of the box. EAS Build compiles in the cloud (no Mac needed to ship). Over-the-air updates for instant fixes. Used in production by Shopify, Discord, Coinbase.

Considered — Bare React Native (no Expo)

More native control, but you hand-wire builds, navigation, notifications, and every native module, and you need a Mac in the loop. Expo's config plugins already expose what we need; bare adds cost for no benefit here.

Considered — Swift / SwiftUI (fully native)

The most "Apple-native" ceiling, but it throws away Android entirely and doubles the surface to build against your ~470-endpoint backend. Your stated goal is one experience across platforms — native-only fights that.

Considered — Flutter

Excellent performance, but a Dart ecosystem separate from your JS/TS world, and its own rendering (not native UIKit components). React Native shares mental models with your existing web JS and hits native components directly.

Trade-offs we accept (and how we mitigate them)

- **Not 100% native widgets.** RN renders real UIKit views, but a handful of bleeding-edge iOS controls need a bridge. Mitigation: Expo's native modules cover our needs; we can drop to a native module later for any specific control.
- **New-Architecture maturity.** We enable the RN New Architecture (Fabric/TurboModules) which is now default and stable in SDK 53. Mitigation: pin the SDK, test each dependency via `expo-doctor`.
- **Heavy media.** Base64-in-Postgres (a backend constraint noted in the audit) caps media quality. Mitigation is a backend concern (object storage + CDN) — the app is built to consume signed URLs the moment they exist.

> **Long-term maintainability.** Expo's yearly SDK cadence, typed routes, and EAS pipeline mean a small team (or you + AI) can maintain iOS and Android from one TypeScript codebase — the same reason Shopify unified 86% of its app and Discord shares 98% across platforms.

ARCHITECTURE DECISION 02

Repository strategy — a dedicated `atwe-mobile` repo

You asked how the professionals structure this. The headline reason teams put a mobile app in a monorepo is to **share React code between a web app and the mobile app.** That benefit does not exist for Atwe — your web frontend is one hand-written vanilla-JS file, not React. So the usual monorepo win evaporates, and a clean dedicated repo is the right structure.

What the big platforms actually do

- **Discord** shares ~98% of code across iOS/Android; **Shopify** hit 86% unification and deleted ~1.8M redundant native lines migrating to React Native (2025); **Coinbase** consolidated onto one RN codebase.
- That cross-platform sharing comes from **React Native/Expo itself** — one codebase → both platforms — **not** from a monorepo.
- Expert guidance (Expo, Turborepo) is consistent: **don't add monorepo complexity until you feel the code-sharing pain.** A monorepo pays off when one schema change must update API + a React client together.

Why a separate repo fits Atwe

- **No web↔mobile React sharing to lose** — the web app isn't React.
- **Clean deploys** — Railway watches the backend repo; mobile commits never trigger a backend rebuild, and EAS gets its own CI.
- **Android comes free anyway** — from Expo, no monorepo needed.
- **Simplicity** for a solo founder + AI: one focused repo, one pipeline.

The bridge to your backend: a typed API contract

The one thing worth sharing between backend and app is the **shape of the API**. The plan: describe the existing Express routes with an **OpenAPI** document, generate **TypeScript types** from it, and vendor those into the mobile repo (seeded already in `src/api/types.ts`). The app then consumes all ~470 endpoints type-safely, and a breaking API change surfaces as a compile error — without coupling the two repositories.

> **Where the code lives right now.** A brand-new GitHub repo couldn't be created from the build environment, so the foundation is committed under `atwe-mobile/` in the backend repo. It is fully self-contained; the README documents a one-command `git subtree split` to lift it into the standalone `atwe-mobile` repo whenever you create it.

CODEBASE

Project structure

A clean, layered separation — UI never talks to the network directly; it goes through hooks → services → the typed API client. This is the professional organization you asked for (UI · business logic · API · navigation · state · utilities · components · hooks · services · types · constants · assets).

```
atwe-mobile/
├─ app/                      expo-router routes (file-based NATIVE navigation)
│  ├─ _layout.tsx            providers + auth gate + root native stack
│  ├─ (auth)/login.tsx       real sign-in — handles the 2FA challenge
│  └─ (tabs)/                the five worlds
│     ├─ _layout.tsx         native tab bar (blur material + haptics)
│     └─ index · beam · engine · ai · profile
├─ src/
│  ├─ theme/                 tokens.ts (ported from web CSS) + ThemeProvider
│  ├─ api/                   client.ts · config.ts · sse.ts · types.ts
│  ├─ auth/                  storage.ts (Keychain) · AuthProvider.tsx
│  ├─ components/            Text · Button · Screen · … (themed primitives)
│  ├─ hooks/                 useFeed, useChat, … (React Query wrappers)   [grows per phase]
│  ├─ services/              feature logic (wallet, orders, …)            [grows per phase]
│  ├─ features/              one folder per world's screens+parts         [grows per phase]
│  ├─ constants/             worlds.ts, config
│  └─ lib/                   queryClient.ts, utilities
├─ assets/                   icon / splash (replace placeholders)
├─ app.json · eas.json       Expo + build config
└─ tsconfig · babel · eslint · prettier
```

Layering rules (enforced by convention + review)

- **Screens (app/, features/)** render and call hooks. They never `fetch` directly.
- **Hooks** wrap React Query around **services**; they own caching keys and optimistic updates.
- **Services** hold feature logic and call the **api** client; pure, testable, no React.
- **api** is the only place that knows URLs, the token, and error shapes.
- **theme** is the only place with colors/spacing; components read tokens, never hex.

ARCHITECTURE

Data, state & realtime

Three layers, each with a clear job — so screens stay simple and data stays consistent across the app and across devices.

Server state — TanStack Query (React Query)

Every backend read is a query; every write a mutation. Caching, background refetch, retries, and optimistic updates are centralized. Query keys mirror the API (e.g. `['feed','foryou']`), so the SSE layer can surgically invalidate exactly what changed.

Realtime — the SSE client (built)

Mirrors the web rt layer: mint a short-lived stream token, connect to `/api/rt/stream`, parse events (msg · typing · presence · notif · wallet · order · call …). On an incoming event we update the relevant React Query cache in place — no refetch. iOS kills backgrounded sockets, so we reconnect on AppState 'active' / network-online (the web app's `rtResync` pattern).

Local/UI state — Zustand + Context

Small, fast stores for ephemeral UI (composer draft, theme, auth). Auth token lives in the Keychain, not in a store. No heavyweight global store — server state belongs to React Query, not Redux.

Offline & resilience

- **Read cache** — React Query's cache is persisted so the last-seen feed/chats render instantly on cold start, then revalidate.
- **Queued actions** — sends carry a client id (the web idempotency pattern), so a retry after a drop can never duplicate. An offline banner mirrors the web behavior.
- **Every API failure is handled** — the client maps 401/403/429/5xx/timeout to friendly, brand-safe messages; the user is never left wondering.

EXPERIENCE

Navigation map & design system

The five worlds — unchanged information architecture

WORLD	ROUTE	WHAT IT HOSTS (BUILT OUT PER PHASE)
Home	`index`	The feed — For You · Following · Circles · Collections; posts, stories, compose.
Beam	`beam`	Messaging & calls — chats, groups, communities, calls, stories, Group Cloud.
Engine	`engine`	Discovery & commerce — search, marketplace, jobs, services, businesses.
Atwe AI	`ai`	The assistant that acts — chat, drafting, analysis, confirm-gated agent actions.
Profile	`profile`	The "Me" hub — profile, wallet, store, settings, everything yours.

A native tab bar switches worlds; each world is its own native stack, so deep pushes (a post, a chat, a listing) get the edge-swipe back gesture and native transitions automatically. Deep links (`atwe://` and universal links from atwe.com) route to the right world.

Design system — ported from the web, elevated for iOS

- **Black** theme (default) and **Light** — exact web palettes.
- **Accent blue = identity only** (links, active tab, AI, verified).
- **White = the one action** per screen ("white acts, blue identifies").
- The neutral **silver verified seal** — never blue.
- **Liquid Glass** — translucent blur on the tab bar, sheets and headers, used sparingly.
- **Dynamic Type** — the type scale respects the user's text-size setting (capped so layout holds).
- **Motion** — Reanimated for subtle, confident transitions; full reduced-motion support.
- **Dark/Light** — system-aware, instant switch, no white flash (native root bg synced).

NON-FUNCTIONALS

Security, notifications & performance

Security

- **Token in the Keychain** — the 30-day bearer lives only in `expo-secure-store` (Secure Enclave-backed), never in plaintext storage. (Built.)
- **Face ID / Touch ID** — `expo-local-authentication` gates app-open and protected sections (Wallet, Storefront), matching the web app-lock model.
- **Apple Sign In** — added when we wire OAuth completion; required by App Review if we offer other social logins.
- **Global 401 → session drop**; the reset flow stays on the emailed link (no in-app token escalation). Certificate/TLS via the OS.

Notifications (architected now, switched on later)

- **APNs via Expo Notifications** — the app already declares the entitlement and requests a device token. The backend's web-push infrastructure extends to an APNs provider; the device registers its token to a new endpoint. Until then, in-app SSE delivery works while open.
- Notification categories map to the existing per-category preferences and quiet-hours already in the backend.

Performance budget

- **Lists** — feeds/chats use FlashList-style virtualization; a bounded window (the web app's DOM-cap lesson) keeps memory flat on endless scroll.
- **Images** — `expo-image` with disk caching, lazy for off-screen, eager for the focused item.
- **Re-renders** — server state in React Query (not prop-drilled), memoized rows, stable keys. Fast launch via the native splash held only through token bootstrap.
- **Network** — request de-dup + caching in React Query; SSE updates in place instead of refetching.

DELIVERY

Phased build roadmap

Each phase ships a runnable, testable slice — you can put every phase on your phone. Tags: DONE shipped in this foundation · NEXT · LATER.

0 **Foundation** DONE

Project + design tokens + typed API/SSE clients + secure auth + five-world navigation + real login & Profile.

1 **Core shell & account** NEXT

Onboarding, signup, settings, theme/accent, account-privacy, notifications list, deep-link routing, offline + push registration.

2 **Home (feed)** NEXT

For You/Following/Circles/Collections, post card, infinite scroll, compose (text/photo/poll), post detail, likes/reposts/bookmarks, stories tray.

3 **Beam (messaging)** NEXT

Conversation list, thread, send (text/photo/voice/meta-cards), reactions/edit/reply, typing/presence/read via SSE, groups, then calls.

4 **Engine (discovery & commerce)** LATER

Search (all scopes), marketplace + listing detail + checkout, jobs board + apply, services/businesses directory, cart/orders.

5 **Atwe AI** LATER

Assistant chat, composer/selection assists, digest, and the agent confirm-card action flow reusing existing routes.

6 **Profile & money** LATER

X-style profile + Me hub, wallet (send/request/pots), storefront management, business tools, analytics.

7 **Polish & App Store** LATER

HIG audit, VoiceOver pass, motion tuning, Apple Pay via Stripe, Apple Sign In, TestFlight beta → submission.

> **Sequencing logic.** Worlds are built in the order that most quickly gives you a demoable, dogfoodable app: the account shell first, then the two worlds users touch daily (Home, Beam), then commerce (Engine), then AI and the deep Profile/money surfaces, then the App-Store polish pass. Each phase reuses backend endpoints that already exist and are already live.

SHIP

App Store readiness & what's already built

Path to the App Store

- **EAS Build** profiles (development / preview / production) are configured in `eas.json` — cloud builds, no Mac required.
- **TestFlight** — `eas submit` uploads to App Store Connect for internal + external beta before public release.
- **Review readiness** — privacy usage strings (camera/photos/mic/Face ID/location) are declared; a privacy "nutrition label", account-deletion path (already in the backend), and Apple Sign In (if other logins ship) close the common rejection reasons.
- **OTA updates** — EAS Update lets us push JS fixes between store releases.

Delivered in this foundation build (verifiable now)

PIECE	FILE(S)	STATE
Design tokens ported from web CSS	`src/theme/tokens.ts`	✓ built
System-aware theming	`src/theme/ThemeProvider.tsx`	✓ built
Typed REST client (+ 401/timeout handling)	`src/api/client.ts`	✓ built
Realtime SSE client (reconnect-safe)	`src/api/sse.ts`	✓ built
Keychain token storage	`src/auth/storage.ts`	✓ built
Auth bootstrap + login (2FA challenge)	`src/auth/AuthProvider.tsx · app/(auth)/login.tsx`	✓ built
Five-world native tab bar (blur + haptics)	`app/(tabs)/_layout.tsx`	✓ built
Real Profile screen (live account + theming)	`app/(tabs)/profile.tsx`	✓ built
Themed primitives (Text/Button/Screen)	`src/components/*`	✓ built
EAS build config + README + assets	`eas.json · README.md · assets/*`	✓ built

To run it today: from `atwe-mobile/ → npm install && npx expo install --fix && npm start`, scan the QR with Expo Go on your iPhone, and sign in with a real account. The Profile tab renders your live Atwe account over the existing backend — the whole auth → API → realtime → render loop, proven end to end.